

Using UEFI on Linux

The Unified Extensible Firmware Interface (UEFI) is a big improvement over BIOS-based bootstrapping of computers. You can finally boot from 2TB+ hard drives natively, and the firmware actually jumps into long mode before booting the operating system, making the job of the OS loader much easier.



UEFI is nice. I've never had a better time managing a multi-boot environment, dual booting Windows 8 and Arch Linux on a UEFI machine, as compared to any BIOS-based machine. If you're curious about trying out UEFI, seriously want to take ownership of your Secure Boot system, or achieve anything in between, read on.

UEFI 101: The basics

Before tinkering with UEFI firmware, one needs to understand how UEFI really works. It is fundamentally different from BIOS. To use UEFI, the disk must be partitioned in a different way from a BIOS-based system. UEFI requires your disk to be partitioned using the GPT (GUID Partition Table) partitioning scheme.

GPT has significant advantages over an MBR-based system. In GPT, all partitions are treated as equal—there are no primary or extended partitions. GPT abandons CHS addressing completely, relying on LBA instead. A GPT-

partitioned drive can hold up to 128 partitions. All GPT partition types are GUIDs (hence the G in GPT).

UEFI extends the concept of the Active Partition (in the MBR scheme) into something called the (U)EFI System Partition (the ESP). The ESP must be formatted with FAT32, and have a GUID of `C12A7328-F81F-11D2-BA4B-00A0C93EC93B`. Unlike BIOS, UEFI does not use a boot sector. Instead, by default (for 64-bit firmware, which ships in almost all computers nowadays), it loads the file located at `/EFI/Boot/bootx64.efi` in the ESP and executes it.

UEFI firmware can be augmented with drivers, and they can and do store boot information in a non-volatile RAM. This information can contain, among other things, a list of other EFI programs that it can load.

Operating systems use this piece of technology to store their bootloaders as EFI executables in a separate file in the ESP. They then configure the firmware to load their file, by default. For example, the Microsoft UEFI Loader for